

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

35 - Building a Datapath

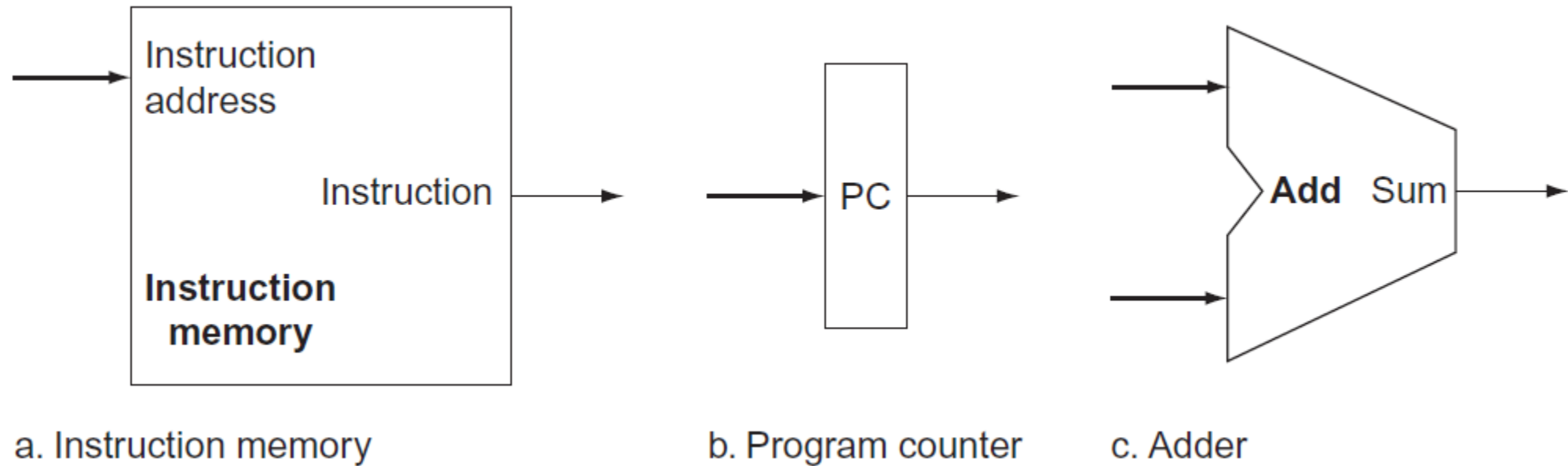
Introduction

- A reasonable way to start a datapath design is to examine major components required to execute each class of MIPS instructions
- Start at top by looking at which **datapath elements** each instruction needs
- When we show datapath elements, we will also show their control signals
- Use abstraction in this explanation, starting from bottom up

Datapath Elements

- **Memory unit** to store the instructions of a program and supply instructions given an address
- **Program counter (PC)**, is a register that holds the address of the current instruction
- **Adder** to increment PC to the address of the next instruction
 - Can be built from ALU by wiring control lines so that control always specifies an add operation
 - Drawn as ALU with label **Add**, to indicate that it has been permanently made an adder and cannot perform other ALU functions

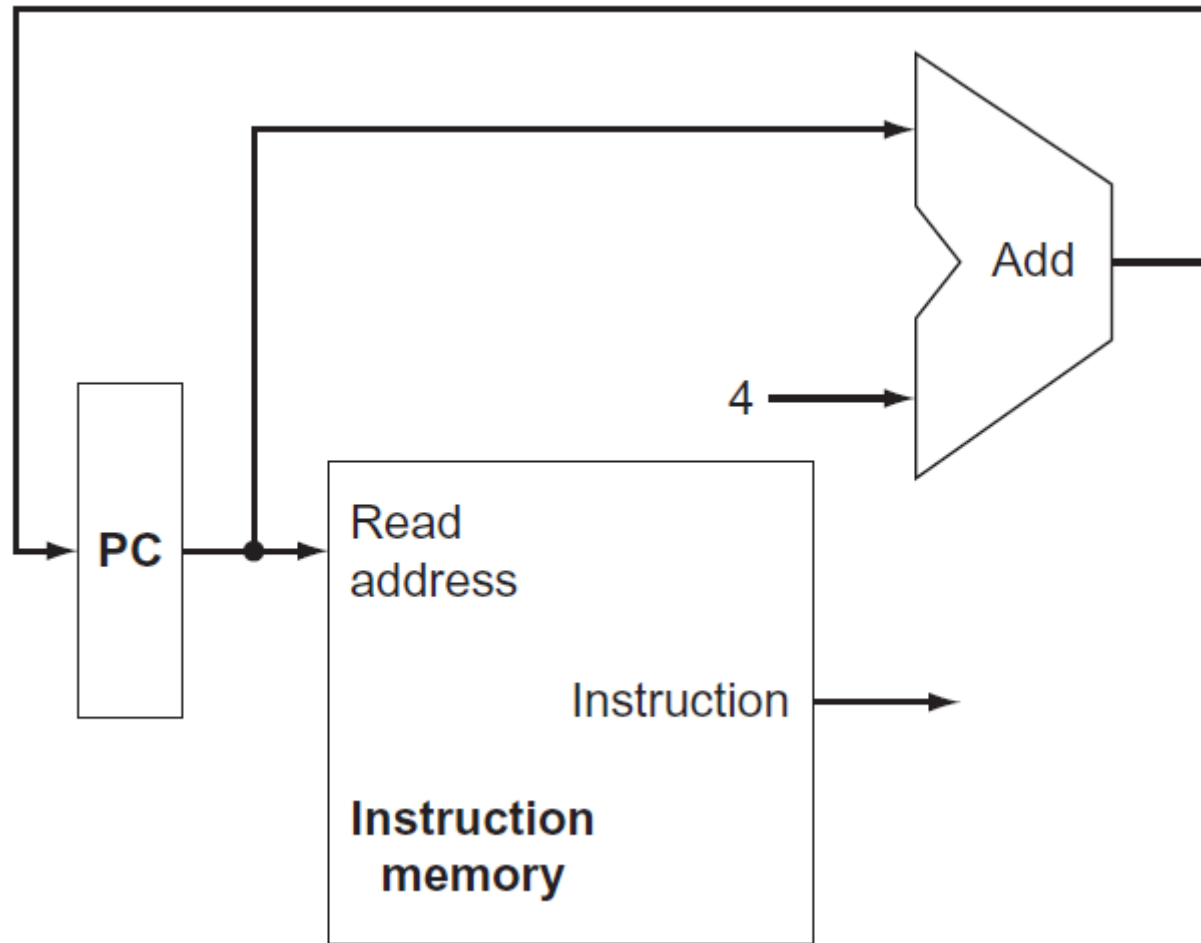
Datapath Elements



Instruction Fetch

- To execute instruction, must start by **fetching** it from memory
- To prepare for executing next instruction, must also **increment PC** so that it points at the next instruction, 4 bytes later
- Figure shows how to combine the three elements from to form datapath that fetches instructions and increments PC to obtain address of the next sequential instruction

Instruction Fetch



Register File

- Processor's 32 general-purpose registers are stored in structure called **register file**
 - Register file is a collection of registers in which any register can be read or written by specifying the number of the register in the file
 - Register file contains register state of the computer
 - In addition, will need an ALU to operate on values read from registers

Read/Write Registers

- R-format instructions have three register operands, so for each instruction we will need to
 - read two data words from register file
 - write one data word into register file
- For each data word to be read from registers, we need
 - input to register file that specifies *register number* to be read
 - output from register file that will carry value that has been read from registers

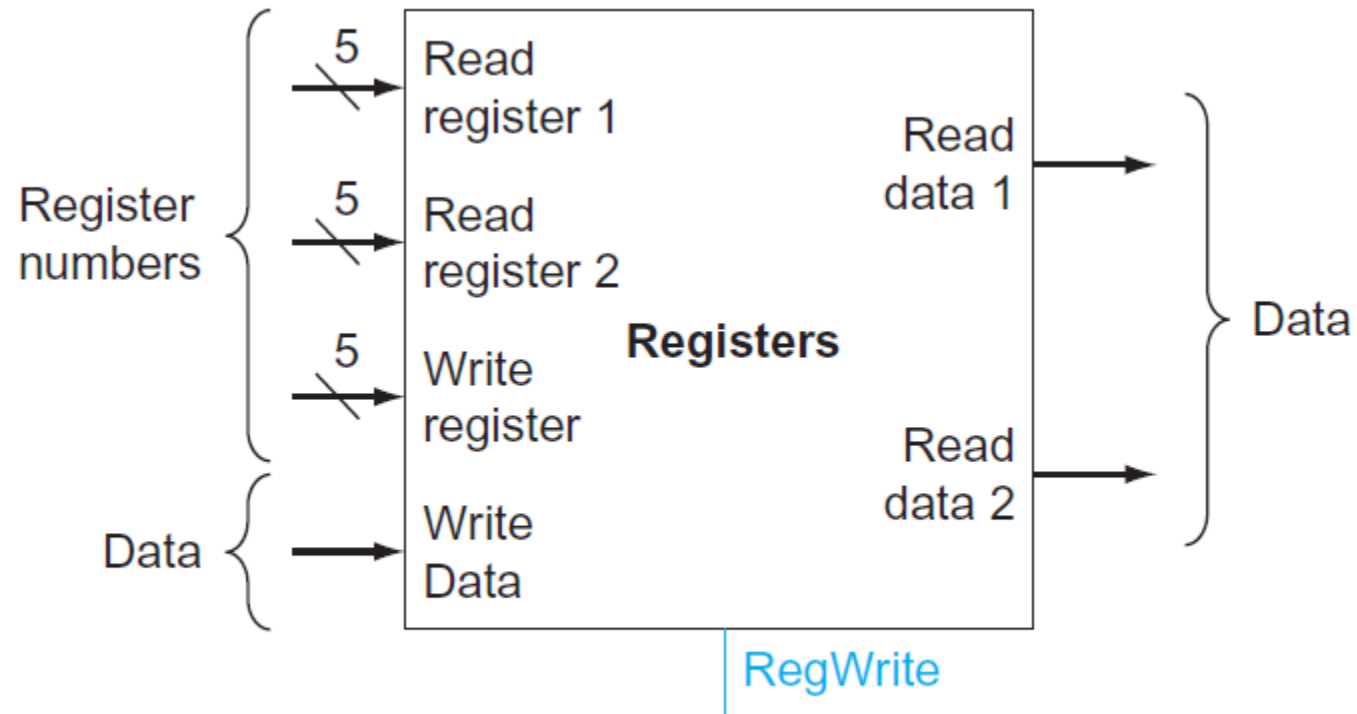
Read/Write Registers

- To write a data word, we will need two inputs
 - one to specify register number to be written
 - one to supply *data* to be written into the register
- Register file always outputs contents of whatever register numbers are on Read register inputs

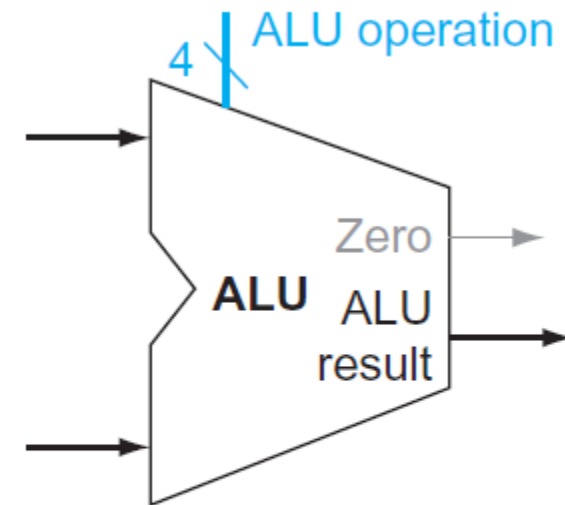
Read/Write Registers

- Writes, however, are controlled by *write* control signal, which must be *asserted* for a write to occur at the clock edge
- Need a total of four inputs (three for register numbers and one for data) and two outputs (both for data)
- Register number inputs are 5 bits wide to specify one of 32 registers ($32 = 2^5$), whereas data input and two data output buses are each 32 bits wide

Read/Write Registers



a. Registers



b. ALU

Read/Write Registers

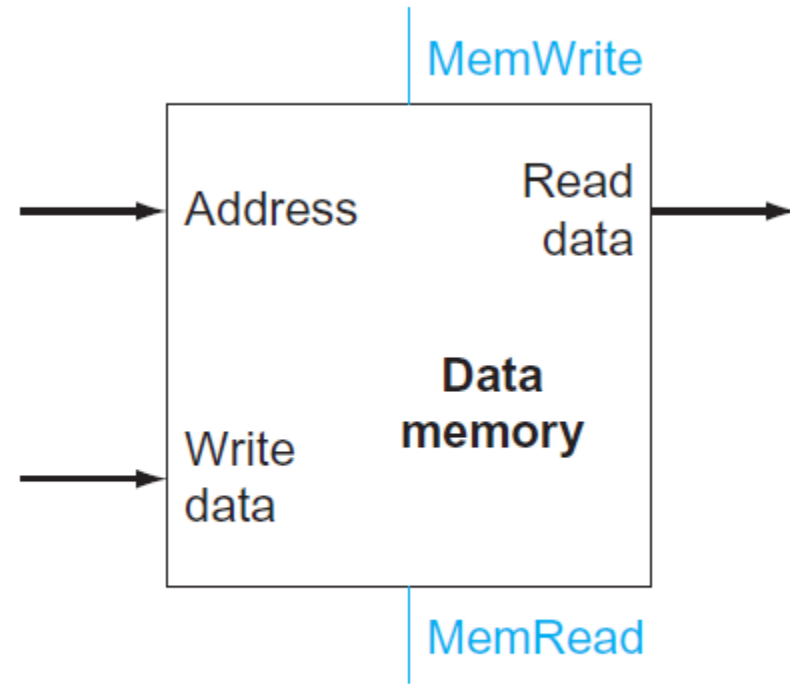
- ALU takes two 32-bit inputs and produces a 32-bit result, as well as a 1-bit signal if result is 0
 - 4-bit control signal of ALU

Load and Store Implementation

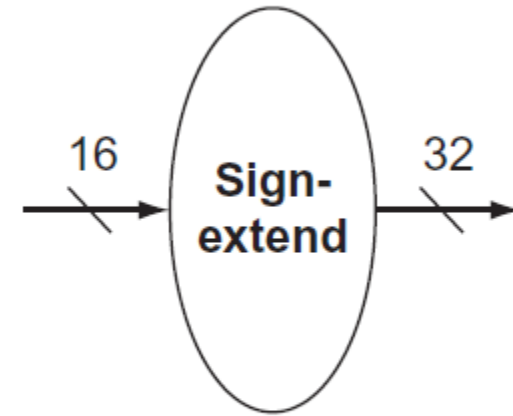
- Consider **lw** and **sw** instructions
 - Compute memory address by adding base register to 16-bit signed offset field contained in instruction
 - If instruction is store, value to be stored must also be read from the register file
 - If instruction is load, value read from memory must be written into register file in the specified register
- Thus, we will need both **register file** and **ALU**

Load and Store Implementation

- In addition, we will need a unit to **sign-extend** the 16-bit offset field in the instruction to a 32-bit signed value, and a **data memory unit** to read from or write to
- Data memory must be written on store instructions; hence it has
 - read and write control signals
 - address input
 - input for data to be written into memory



a. Data memory unit



b. Sign extension unit

Branch Implementation

- The `beq` instruction has three operands, two registers that are compared for equality, and a 16-bit offset used to compute the `branch target address` relative to branch instruction address
- Its form is `beq $t1,$t2,offset`
- To implement this instruction, we must compute branch target address by adding sign-extended offset field of the instruction to PC

Branch Implementation

- There are two details in the definition of branch instructions (see Chapter 2) to which we must pay attention:
- ■ The ISA specifies that the base for the branch address calculation is the address of the instruction following the branch. Since we compute $PC + 4$ (the address of the next instruction) in the instruction fetch datapath, it is easy to use this value as the base for computing the branch target address.
- ■ The architecture also states that the offset field is shifted left 2 bits so that it is a word offset; this shift increases the effective range of the offset field by a factor of 4